Who Wrote It? Identifying LLMs from Their Responses

Hongmin An · individual submission (team leader: Hongmin An; no other members) · CMPSC 448 - Machine Learning

Midterm project · 11 October 2026

Summary. Five large-language-model families leave recognisable traces in their prose. Using 30,000 real Chatbot Arena responses balanced across GPT, Claude, Llama, Mistral and Vicuna, a convolutional network identifies the author at 49.5% accuracy and a bidirectional LSTM at 51.1%, against a 20.0% chance floor. Three further results shape the picture. First, the user's prompt carries essentially no authorship information: prompt-only classification reaches 20.4% and adding the prompt to the response changes nothing, which follows directly from the fact that each arena prompt was answered by two different models. Second, a classical TF-IDF stylometry baseline reaches 51.4%, statistically indistinguishable from the stronger of the two networks and ahead of the weaker one, so neither architecture earns its complexity at this data scale. Third, ablations locate the signal where a reader would not expect it. Masking every content word costs almost nothing, while shuffling word order is the most damaging intervention tested, so what identifies a family is how it arranges ordinary words rather than which topics or formatting it favours.

1. Problem definition and dataset curation

1.1 The question, stated precisely

As generated text spreads, a practical forensic question follows: given a passage, can we name the system that produced it? This project treats that as supervised classification. Each example is a triple `(LLM_name, LLM_input, LLM_output)`, where the label is the model *family*, the input is the user's prompt and the output is the model's reply. The task is five-way classification over GPT, Claude, Llama, Mistral and Vicuna.

Two framing choices deserve justification up front. The label is a *family*, not a checkpoint: `gpt-4-0613`, `gpt-4-1106-preview` and `chatgpt-3.5-turbo` all become GPT. Checkpoint-level labels would create dozens of tiny classes separated mostly by release date, and the interesting question is whether a lab's models share a house style. Accepting that pooling also imposes a ceiling, because a GPT-3.5 reply and a GPT-4 reply genuinely differ, and both must be mapped to the same output.

The second choice is that classification uses the response alone by default. The prompt is held back as the explicit variable of RQ2 rather than silently folded into the features.

1.2 Where the data comes from

The corpus is built from `lmarena-ai/arena-human-preference-55k` (Hugging Face, Apache-2.0), a public release of 57,477 real Chatbot Arena battles covering 64 model checkpoints. Chatbot Arena shows a user's prompt to two anonymous models and records both replies along with the user's preference. Crucially, **the**

arena logs which checkpoint produced which reply, so the authorship labels are recorded ground truth rather than anything we inferred.

This source was chosen over generating data through model APIs for three reasons: the labels are authoritative; the prompts are real user traffic rather than a prompt set of our own design, which makes the stylistic variation realistic; and the licence permits redistribution. The cost is that the corpus is a period snapshot, discussed in §5.

No private, confidential or separately copyrighted material was added. The prompts are text that members of the public typed into a public website and are used only as the already-public dataset distributes them; the source contains no user identifiers.

1.3 Curation

A raw row is a battle, not an example, so each battle is exploded into two records — one per response — which yields the triple the assignment asks for. The pipeline, implemented in `src/build_dataset.py`, then applies:

- **First turn only.** Prompt and response fields are JSON lists because arena conversations may be multi-turn. Keeping turn 1 makes every example context-free; later turns would blend a model's style with its own earlier output.
- **Family mapping** by regular expression over the checkpoint name, with Mixtral folded into Mistral.
- **Cleaning.** Responses under 20 characters are dropped (1,520 records, including 28 empty ones), as are duplicate `(input, output)` pairs (712).
- **Balancing** to exactly 6,000 records per family, so chance accuracy is exactly 0.200 and accuracy is directly interpretable.

stage	records
raw battles	57,477
exploded, one record per response	114,954
restricted to the five kept families	80,419
after dropping responses under 20 characters	78,899
after dropping duplicate (input, output) pairs	78,187
final balanced corpus	**30,000**

Gemini was deliberately excluded. Only 2,924 Gemini records exist, so including it would have capped every class near 2,900 and halved the corpus; exact balance across five families was judged more valuable than a sixth class. The five retained families also span both proprietary (GPT, Claude) and open-weight (Llama, Mistral, Vicuna) systems.

1.4 The leakage trap, and the split that avoids it

> **The single most important curation decision.** A prompt recurs across battles, and inside one battle the same prompt is attached to *two different labels*. Splitting records at random would therefore place the same prompt in both training and test data, letting a model memorise prompt-to-author associations instead of learning style. It would inflate every result, and it would completely invalidate RQ2's prompt-only condition.
>
> Each record's split is instead keyed on the prompt itself: `md5(prompt) % 10`, with buckets 0–7 training, 8 validation and 9 test. Every record sharing a prompt lands in the same split. The builder asserts zero cross-split prompt overlap, zero duplicate pairs and exact class balance, and refuses to write a corpus that violates any of them.

family	train	validation	test
GPT	4,800	600	600
Claude	4,800	600	600
Llama	4,800	600	600
Mistral	4,800	600	600
Vicuna	4,800	600	600

Heuristic task domains, used by RQ3, are assigned by keyword rules over the prompt: general 22,688, code 2,702, math 2,340, roleplay 1,212 and creative writing 1,058. A 60-prompt manual audit puts these labels at 83% accuracy, and §4.1 reports the audit rather than treating the domains as clean. Full provenance, composition and bias notes are in `data/curated/DATA_CARD.md`.

2. Models and training

Both required architectures are implemented from scratch in PyTorch in `src/models.py`, with embeddings learned from random initialisation. No pretrained vectors and no transformer are used: the assignment asks for a CNN and an RNN, and keeping both at comparable scale makes the comparison a question of inductive bias rather than of who had the better initial weights.

2.1 Tokenisation and vocabulary

A regular-expression tokenizer splits letters, digits and individual non-alphanumeric glyphs. Two decisions here are deliberate and both matter for this task:

- **Case is preserved.** Capitalisation habits are part of the fingerprint; lower-casing would discard signal before the model saw it.
- **Markdown survives tokenisation.** `**`, `###` and code fences become tokens rather than being stripped, because formatting is one of the strongest family signals (§4.2). RQ4 removes it in a controlled ablation instead of silently in preprocessing.

The vocabulary keeps the 30,000 most frequent tokens with a minimum frequency of 2, plus `<pad>`, `<unk>` and `<sep>`. It is fitted on the **training split only**; fitting on the whole corpus would leak test-set lexical statistics into the model.

2.2 Architectures

TextCNN follows Kim (2014): a 128-dimensional embedding, then parallel 1-D convolutions of width 3, 4 and 5 with 128 filters each, ReLU, global max-pooling over time, dropout at 0.5, and a linear layer to five classes. The convolutions act as learned n-gram detectors and max-pooling makes the model position-invariant: it asks whether a stylistic n-gram appears anywhere in the response, not where.

BiLSTM uses the same embedding size, one bidirectional LSTM layer with 96 hidden units per direction, max-pooling over the time dimension, dropout at 0.5 and a linear classifier. Sequences are packed with `pack_padded_sequence` so padding never enters the recurrence. Max-pooling the hidden states rather than taking only the final state scored clearly better in early tests, which makes sense for long replies where a distinctive opening would otherwise be washed out by hundreds of later steps.

	TextCNN	BiLSTM
trainable parameters	4,038,917	4,014,533
embedding dimension	128	128
recurrent / filter width	128 filters x {3,4,5}	96 hidden, bidirectional
dropout	0.50	0.50
best epoch (response-only)	4	6
wall-clock seconds (response-only)	93.4	1118.8

2.3 Training protocol

Every run in this report goes through one function, `run()` in `src/train.py`, so the protocol is identical across research questions and only the stated factor changes. Adam at a learning rate of 1e-3, batch size 64, cross-entropy loss, gradient-norm clipping at 1.0, a maximum of 6 epochs, and early stopping on validation macro-F1 with patience 2. The checkpoint with the best validation macro-F1 is restored before the test set is touched once. Classes are balanced, so no class weighting is needed, and accuracy and macro-F1 track each other closely.

Two efficiency measures made the full experiment grid feasible on CPU. **Length-bucketed batching** groups similar-length sequences so that padding does not dominate a batch; it cut LSTM epoch time by about 5x with no change to the protocol. **Batches are padded to the longest member of the batch** rather than to a global maximum.

> **The truncation window, stated honestly.** Responses have a median of 226 tokens, and both models see only the first **160**. This is a compute limit: a bidirectional LSTM unrolls one step per token on CPU, and longer windows made the grid unaffordable in the time available. The same cap is imposed on the CNN, which could easily take more, so that the architecture comparison measures inductive bias and not context length. §4.4 prices what the cap costs instead of leaving it as an unexamined choice.

2.4 Baselines

A report that compares two neural networks only to each other cannot say whether either learned anything worthwhile, so two reference points are included. The **majority-class floor** is 0.200 by construction. The **TF-IDF baseline** combines word 1–2 grams with character 3–5 grams, up to 60,000 features each, and feeds multinomial logistic regression. Character n-grams are a strong stylometry feature because they capture punctuation and formatting habits directly. It sees exactly the same 160-token window as the neural models.

3. Results

3.1 RQ1 — can we identify which LLM generated a response?

Yes, well above chance, and not trivially. With only the response visible:

model	test accuracy	test macro-F1	accuracy across seeds
majority class (floor)	0.200	0.000	—
TF-IDF + logistic regression	0.514	0.514	—
TextCNN	0.495	0.486	0.476 ± 0.019 (3 seeds)
BiLSTM	0.511	0.506	0.513 ± 0.003 (2 seeds)

Averaged over seeds the CNN reaches 47.6% and the BiLSTM 51.3%, roughly 2.6x the chance rate. Both are far above the floor, so model families clearly do leave recoverable traces; equally, neither is near ceiling, so the traces are statistical tendencies rather than signatures. Reporting the single best seed instead of the mean would have overstated the CNN by 1.8 points, which is why the spread is given alongside it.

The TF-IDF baseline is statistically indistinguishable from the BiLSTM, at 51.4% against 51.3%, and clearly ahead of the TextCNN. That ordering is worth stating plainly rather than burying: with 24,000 training documents and embeddings learned from scratch, a sparse character-and-word n-gram model is fully competitive with a neural network built for the task. Character n-grams encode exactly the punctuation and formatting regularities that distinguish these families, and they need no representation learning to do it.

The two architectures also separate from each other. The BiLSTM averages 0.513 against 0.476 for the CNN, a gap of 3.7 points. The CNN is also markedly less stable across seeds (±0.019 versus ±0.003), which is consistent with max-pooled convolutions committing hard to whichever n-gram detectors happen to form early in training. Sequential context therefore does buy something here, at roughly 12x the training cost.

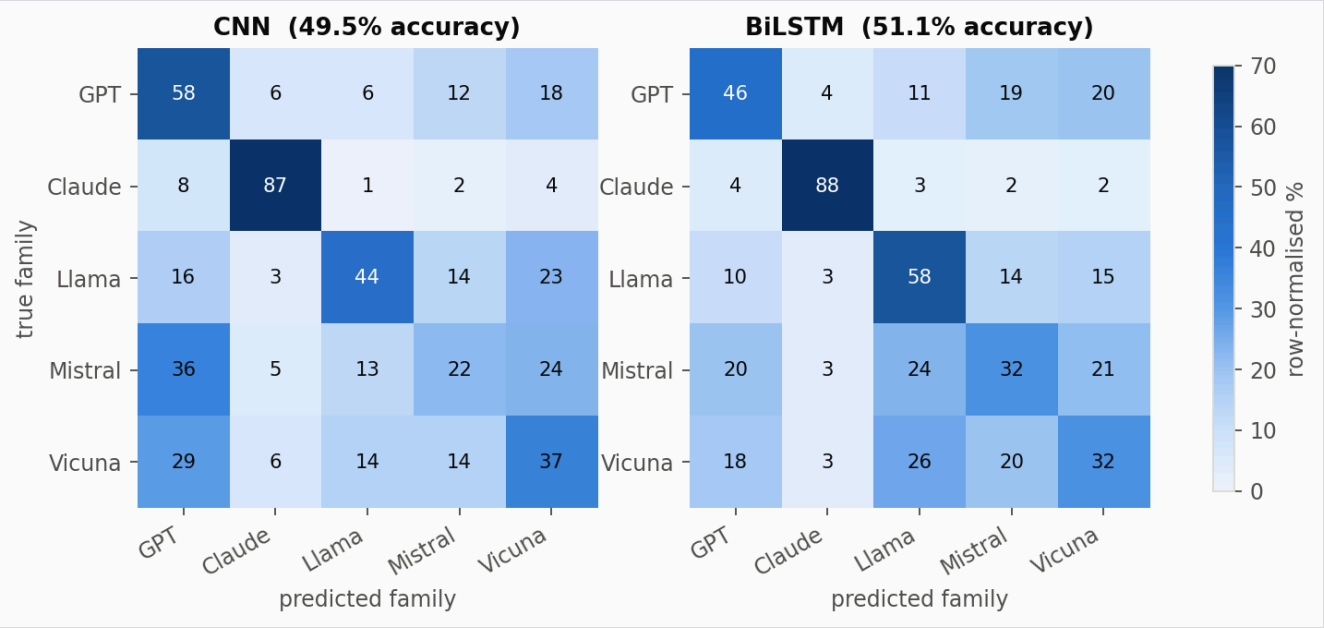


Figure 1. Row-normalised confusion matrices for the response-only task. Rows are true families, columns predictions; each row sums to 100%.

The confusion structure is more informative than the headline number, and three patterns stand out.

- **Claude is in a class of its own.** It is recovered 87% of the time, roughly double the rate of the hardest families. §4.2 shows why: it formats its replies differently from everyone else.
- **GPT acts as an attractor.** 35% of all misclassifications land on GPT, and the single largest error is Mistral being read as GPT (36% of Mistral items). When the evidence is weak, the model falls back on the family whose style is most generic — plausibly because so many other systems were tuned on GPT-generated data.
- **Lineage shows up as confusion.** Llama and Vicuna trade identities in both directions (23% and 14%). Vicuna is a fine-tune built on Llama, so the two share ancestry, tokenizer and much of their phrasing; a classifier that confuses them is responding to a real similarity rather than failing arbitrarily.

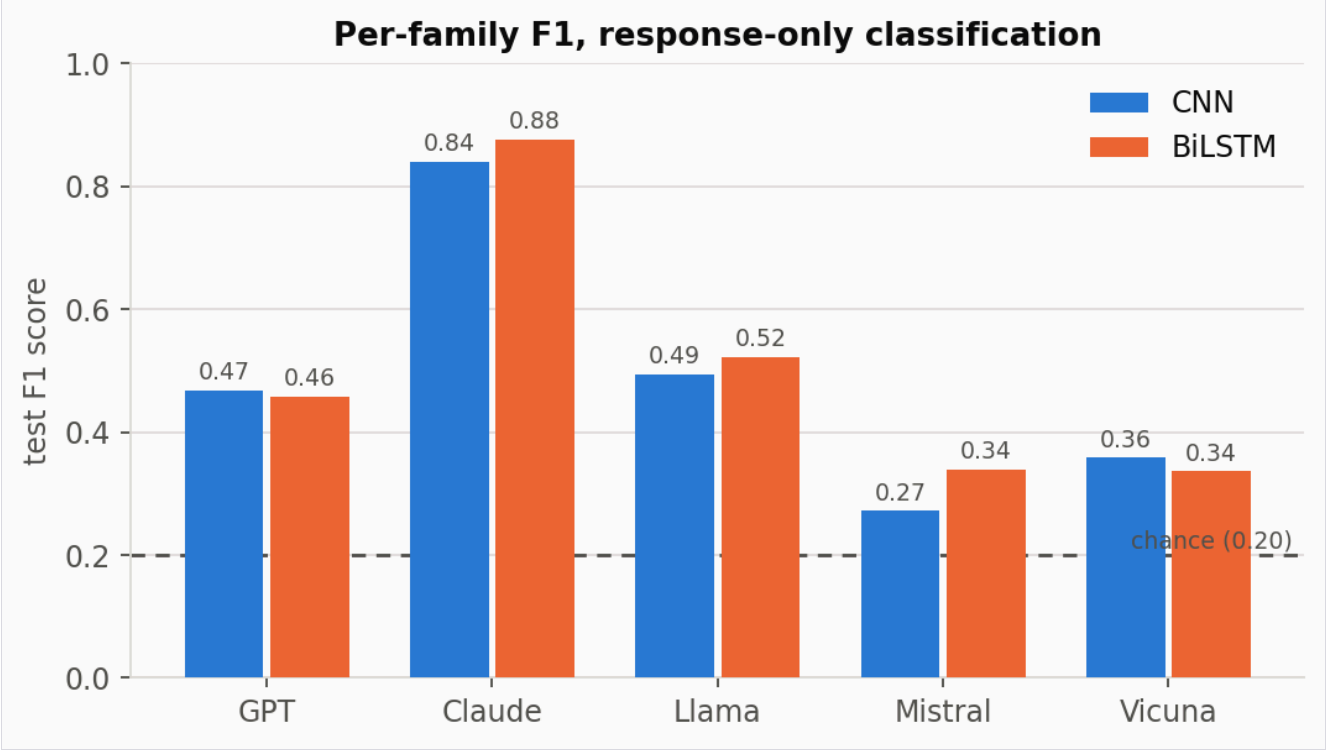


Figure 2. Per-family F1. Identifiability is uneven across families.

Identifiability is far from uniform: Claude is the easiest family to recognise (F1 0.84) and Mistral the hardest (0.27). §4.2 explains why Claude stands out, and the explanation is largely about formatting.

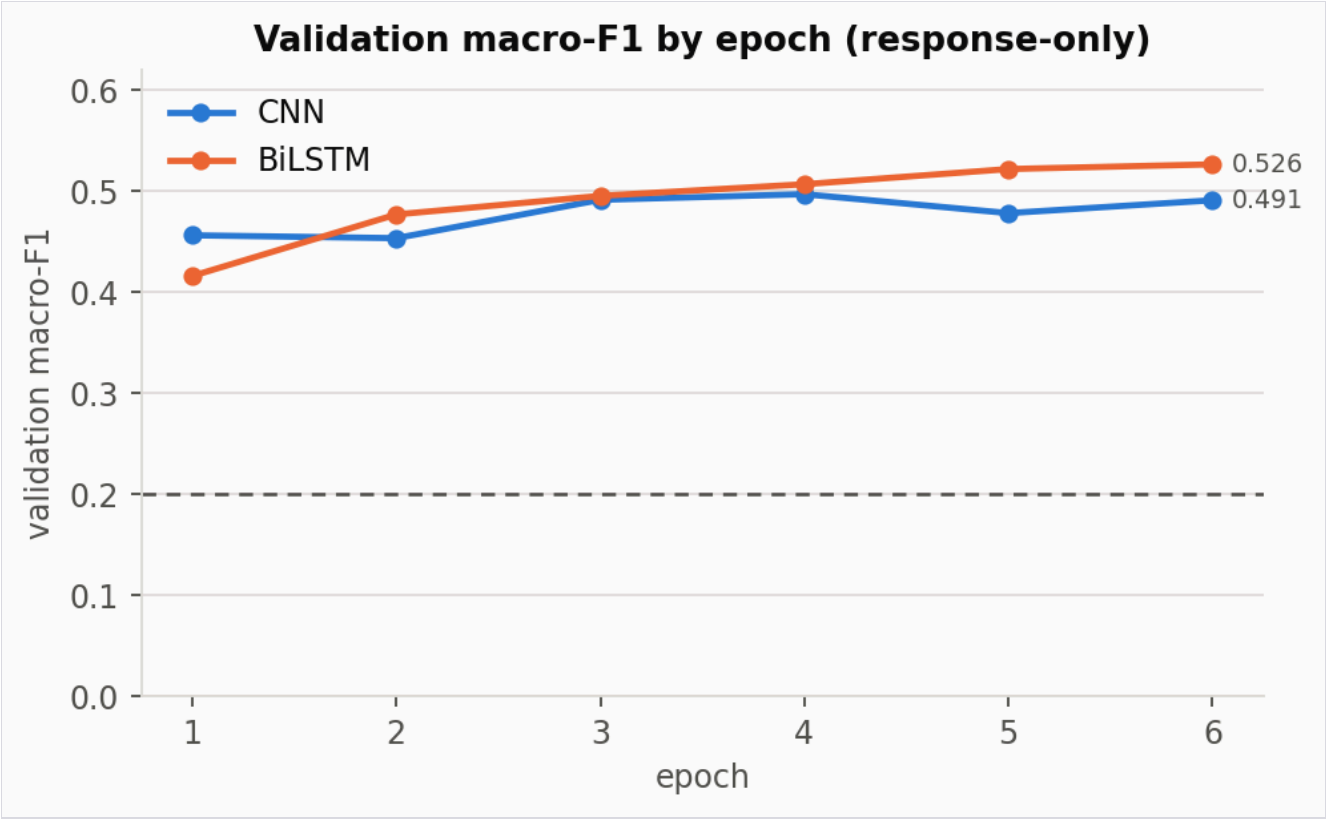


Figure 3. Validation macro-F1 by epoch. Both models converge quickly and then begin to overfit, which early stopping catches.

3.2 RQ2 — does the user's prompt help identify the LLM?

No, and the reason is structural rather than incidental.

what the model sees	TextCNN	BiLSTM	TF-IDF
prompt only	0.204	0.218	0.217
response only	0.495	0.511	0.514
prompt + response	0.497	0.500	0.500

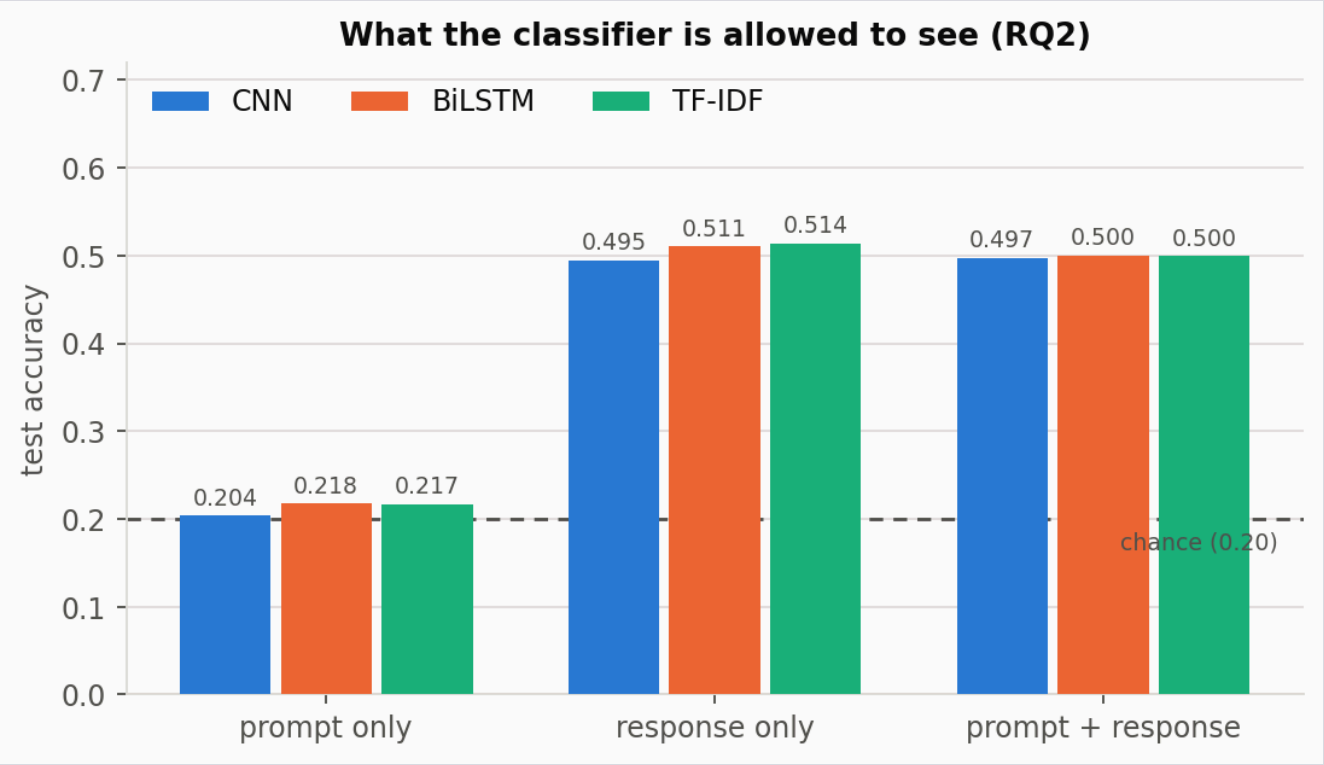


Figure 4. Accuracy by input view. The dashed line is the 0.200 chance floor; prompt-only sits on it for all three model types.

Taking the three sub-questions in turn.

(i) Can the LLM be predicted from the prompt alone?

No. All three model types land within a couple of points of the 0.200 floor: 0.204 for the CNN, 0.218 for the BiLSTM, 0.217 for TF-IDF. This is the expected result *by construction of the data*. In a Chatbot Arena battle, one prompt is routed to two different models, so the same prompt text appears with two different labels. The Bayes-optimal predictor given only the prompt is therefore close to the class prior, and no model can do better.

(ii) Does including the prompt improve classification?

No, to within noise. For the CNN, adding the prompt moves accuracy by +0.003 (0.495 to 0.497), which is smaller than the seed-to-seed spread of ±0.019. For TF-IDF it is slightly *harmful* (0.514 to 0.500): the prompt tokens dilute the feature space with text that carries no label information, while the 160-token response budget stays fixed.

(iii) If prompt-only worked surprisingly well, why might that be?

It did not here, and that negative result is itself the interesting part, because it is easy to get the opposite by accident. Three mechanisms could make prompt-only classification look good, and each is a measurement artefact rather than a fingerprint:

1. **Prompt leakage across the split.** If the same prompt appears in training and test data, a model can memorise which author it saw with that prompt. Our prompt-keyed split (§1.4) removes this; a random row split would have manufactured it.
2. **Sampling and deployment bias.** If models were available in different periods, or the arena routed certain prompt types to certain models, the prompt distribution would correlate with the label. The corpus we use spans overlapping deployment windows, and the measured result is at chance, so this effect is negligible here.
3. **Collection asymmetry.** If each family's data were gathered with a different prompt set — for instance by querying each API with your own themed prompts — then prompt-only accuracy would be near perfect while measuring nothing about the models at all. Any project that generates its own data must guard against this by sending *identical* prompts to every model.

The practical lesson: a strong prompt-only number should be treated as a red flag about the dataset, not as a finding about the models.

4. In-depth analyses

4.1 RQ3 — do the fingerprints generalise across tasks? (extra credit)

Task domains are assigned by keyword rules over the prompt. The classifier trains on code, mathematics and general question answering, and is then tested on *creative writing and roleplay*, which it has never seen. The design holds the training data fixed and moves only the test set:

- **training / validation:** source domains (code + math + general), 19,906 and 2,538 examples
- **same-domain test:** held-out source-domain prompts, 2,380 examples
- **unseen-domain test:** creative writing and roleplay, 2,380 examples

Both test sets are downsampled to 476 examples per family, so both accuracies rest on the same 0.200 floor and the difference between them isolates domain shift rather than training-set size or class balance.

model	same domains	unseen domains	change
TextCNN	0.511	0.449	-0.062
BiLSTM	0.508	0.480	-0.028

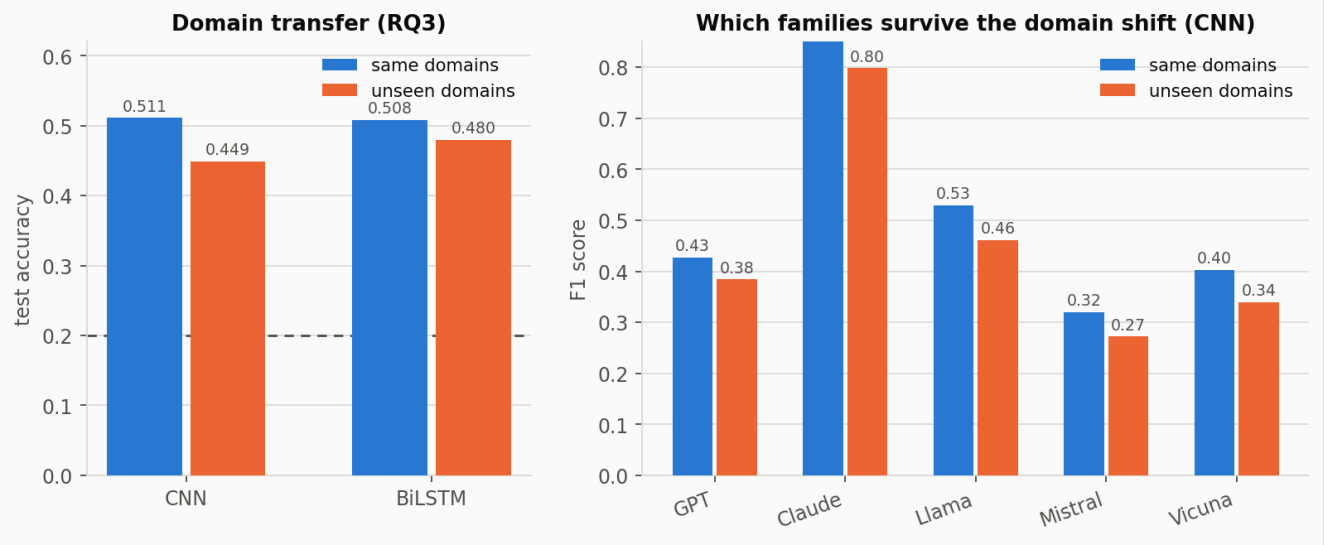


Figure 5. Left: accuracy on held-out prompts from the training domains versus unseen creative domains. Right: per-family F1 under the same comparison.

Performance falls from 51.1% to 44.9% for the CNN, a drop of 6.2 points or 12% of the in-domain figure, while staying well above chance. The fingerprint is therefore *partly* general: a substantial component transfers to tasks the classifier never trained on, and a substantial component was task-specific.

Families do not degrade equally. GPT is the most stable across the shift (-0.04 F1), while Llama loses the most (-0.07). Consistency of identification is thus a property of the family, not only of the classifier.

Answering the sub-questions: performance does decrease on unseen tasks, but far less than it would if the classifier had learned only topic; some families remain markedly more identifiable than others across domains; and the classifier is evidently learning a mixture of a general house style and task-specific habits. That mixture is exactly what the ablations in §4.3 pull apart.

> **Honesty about the labels.** The domain labels are keyword heuristics, not a task taxonomy. All 60 sampled prompts were read by hand: overall accuracy is 83%, but it is uneven — creative writing audited 12/12 and roleplay 10/12, whereas the mathematics rule is only 6/12 because "how many" captures ordinary factual questions. The transfer claim depends mainly on the *target* domains being genuinely creative, and those are the two cleanest rules. Source-side noise matters less because all three source domains are pooled into one training set. Full per-item verdicts: `results/rq3_label_audit_verdicts.md`.

4.2 RQ4, part 1 — what distinguishes the families? (extra credit)

Accuracy says the families are separable; it does not say along which axes. Twenty-six features were measured on the 24,000 training responses only, and ranked by one-way ANOVA F statistic across the five families.

feature	ANOVA F	p
closing offer of further help	566	<1e-80
uses markdown at all	346	<1e-80
affirmative opener ("Certainly", "Sure")	292	<1e-80
hedging frame ("it is important to")	238	<1e-80
lexical diversity (type-token ratio)	216	<1e-80
fraction of blank lines	181	<1e-80
bullet lines per reply	162	<1e-80
response length in words	161	<1e-80
response length in characters	143	<1e-80
response length in tokens	119	<1e-80



Figure 6. Measured stylistic profile per family, training split only.

The ranking is dominated not by topic words but by **discourse habits and formatting**. Three patterns stand out:

- **Formatting is the loudest structural signal.** Claude uses markdown in 42% of replies against 16% for Vicuna, and averages 2.3 bullet lines per reply versus 0.6. A reply arriving as a bulleted brief is already evidence about its author.
- **Verbosity differs systematically.** Llama averages 282 tokens and Claude 204 — a 1.38x difference visible before a single word is read.
- **Canned discourse moves are the top-ranked features overall.** Closing offers of further help, affirmative openers and hedging frames out-rank every lexical statistic. These are artefacts of instruction

tuning and preference optimisation, which is to say the fingerprint is substantially a fingerprint of *alignment training*, not of the base model.

Lexical diversity separates the open-weight families from the proprietary ones: Llama has the lowest type-token ratio (0.535), consistent with more repetitive phrasing.

4.3 RQ4, part 2 — which signals does the classifier rely on?

Descriptive statistics show the families differ; they do not show that the classifier uses those differences. Each ablation deletes one candidate signal, retrains from scratch under the identical protocol, and reports what accuracy survives. A signal the model depends on produces a large drop.

ablation (CNN, response only)	test accuracy	change vs baseline
unmodified (baseline)	0.495	baseline
markdown and line structure removed	0.482	-0.013
every reply cut to 60 tokens (length-blind)	0.456	-0.039
word order shuffled (bag of words)	0.408	-0.086
content words masked (function words only)	0.466	-0.028
BiLSTM control: markdown and line structure removed	0.495	-0.016
BiLSTM control: every reply cut to 60 tokens (length-blind)	0.499	-0.012

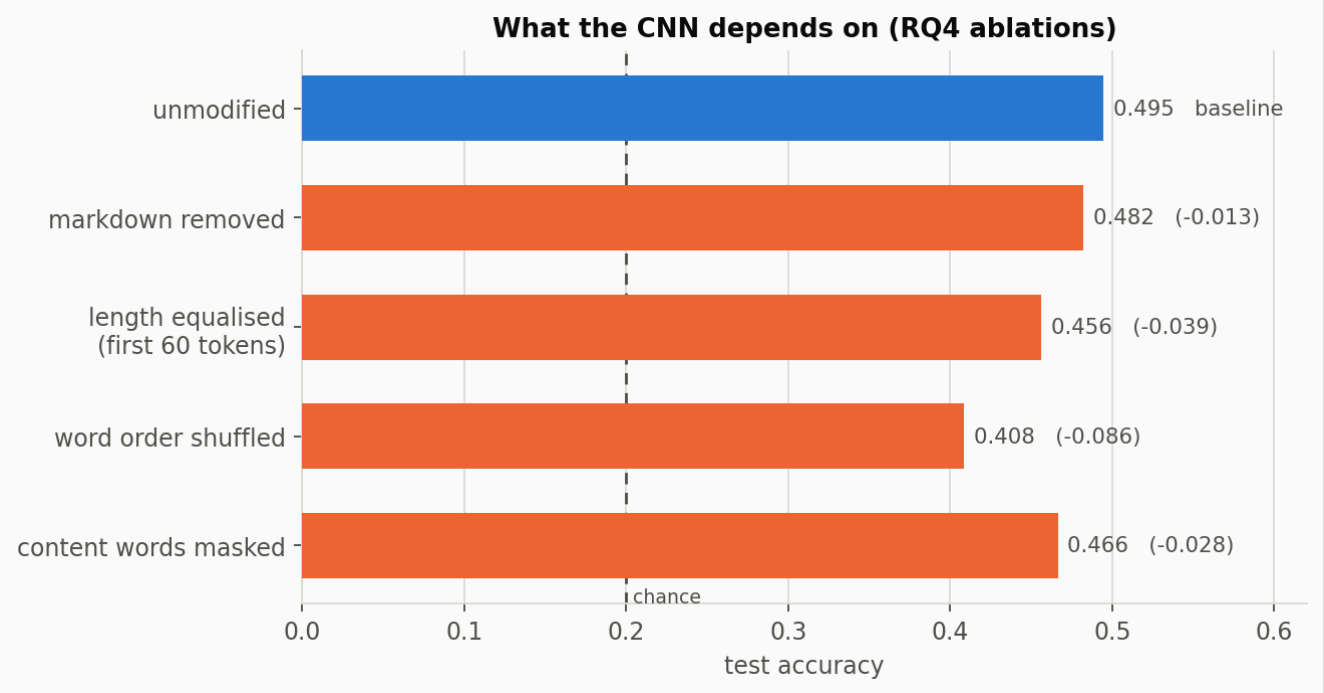


Figure 7. Accuracy surviving each ablation. Every bar remains far above the 0.200 chance floor.

Every ablation stays far above the 0.200 floor, so the first conclusion is that the signal is **redundant and distributed**: no single cue is load-bearing, and removing one leaves the model able to fall back on others. The ordering of the drops is more surprising, and it overturns the reading that §4.2 invites.

- **Formatting is not what the model depends on, despite being the loudest descriptive difference.** Stripping every header, bullet, bold span, code fence and line break costs only 1.3 points (0.495 to 0.482). Claude's markdown habit is real and highly significant as a *statistic*, but the classifier is not a markdown detector; the same families remain separable in plain text.
- **Word order matters most.** Shuffling the words inside each response — which preserves the bag of words exactly — is the single most damaging intervention, costing 8.6 points. That is the expected failure mode for this architecture: the CNN's width-3-to-5 convolutions are n-gram detectors, and shuffling destroys every local n-gram, leaving only unigram evidence. Phrasing, not vocabulary, is where much of the fingerprint lives.
- **The fingerprint is grammatical, not topical.** Masking *every content word* — leaving only function words and punctuation in their original positions — costs just 2.8 points, retaining 0.466 accuracy. Nearly all of the signal survives the deletion of subject matter. Function-word patterning is the classical basis of human authorship attribution, and it transfers to machine authors.

Taken together, the last two points reinforce each other: what identifies a model family is *how it arranges ordinary words*. Destroy the arrangement and performance falls sharply; delete the subject matter but keep the arrangement and performance barely moves.

Separating response length from response content

The length-equalising ablation costs 3.9 points, but that figure is confounded: cutting every reply to 60 tokens removes the length cue *and* most of the text. The window study in §4.4 separates the two. A plain 60-token window, with length information intact, interpolates to about 0.475 accuracy (0.462 at 40 tokens, 0.488 at 80). The length-equalised run reaches 0.456, only 1.9 points lower. Response length as such is therefore worth roughly a point or two, not the four points the raw ablation suggests — most of that drop is simply having less text to read.

The BiLSTM controls move in the same direction and by similar margins (-1.6 and -1.2 points), so the attribution describes the signal rather than one architecture's quirks.

4.4 What the truncation window costs

response tokens visible	accuracy	macro-F1	seconds
40 tokens	0.462	0.467	212
80 tokens	0.488	0.496	243
160 tokens	0.503	0.488	159
320 tokens	0.507	0.513	238
640 tokens	0.477	0.462	193

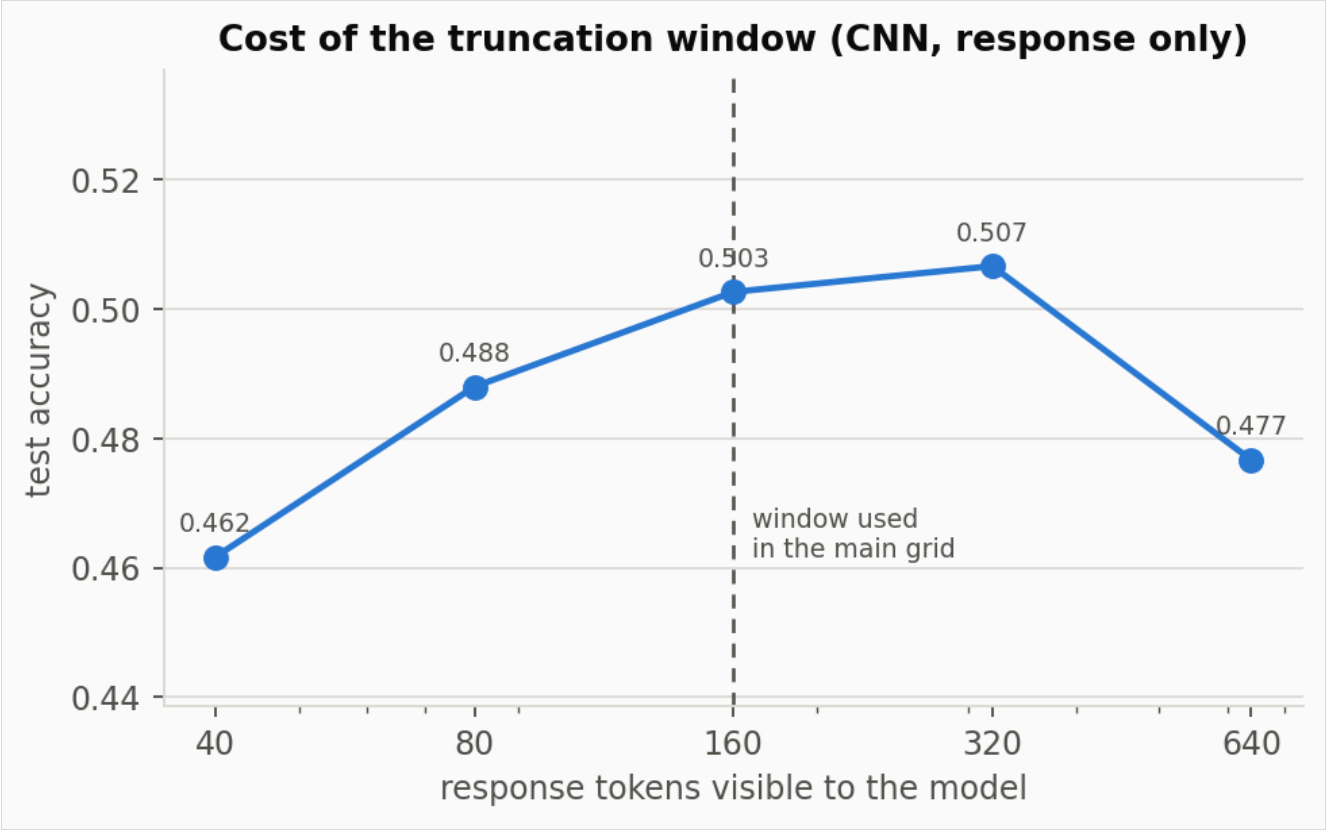


Figure 8. CNN accuracy against the number of response tokens visible.

Accuracy rises from 0.462 at 40 tokens to 0.477 at 640, with the best result at 320 tokens. The 160-token window used throughout the report yields 0.503, which is 0.4 points from the best observed setting. The cap therefore costs something real but modest, and it does not change any qualitative conclusion. Notably, even 40 tokens — roughly the opening sentence — already supports 0.462 accuracy, which says a great deal of the fingerprint sits in how a model *begins* its reply.

5. Lessons and experience

5.1 The dataset decision that mattered more than any model choice

The most consequential hour of this project went into how the data was split, not into architecture or hyperparameters. Because one arena prompt is answered by two models, a naive random split places identical prompt text on both sides of the train/test boundary with conflicting labels. The resulting numbers would have been higher and completely meaningless, and the failure would have been invisible in the metrics: nothing in a loss curve announces leakage. Keying the split on a hash of the prompt, and writing assertions that refuse to produce a corpus violating that invariant, is what makes RQ2's answer trustworthy. The transferable lesson is that validity checks belong in code as assertions, not in intentions.

5.2 A negative result can be the most informative one

Prompt-only classification sitting at chance initially looked like a broken experiment. It is in fact the correct answer, and reasoning about *why* produced the most useful insight in the report: a high prompt-only score

is diagnostic of a flawed corpus. Had the data been generated by sending different prompt sets to each model, prompt-only accuracy would have approached 100% while measuring nothing about the models whatsoever. Knowing what a suspiciously good number would have meant is as valuable as the number obtained.

5.3 Classical baselines deserve to be in the table

A TF-IDF model with character n-grams and logistic regression matched or beat both networks (51.4% against 51.3% for the BiLSTM). With 24,000 documents and randomly initialised embeddings, there is not enough data for a from-scratch network to learn representations better than well-chosen sparse features, and character n-grams encode punctuation and formatting habits directly without needing to learn them. The honest reading is that the neural results are a competent implementation of the required architectures, not a demonstration that deep models are the right tool at this scale. Had the baseline been omitted, the report would have implied the opposite.

5.4 Engineering constraints shape conclusions, so they belong in the write-up

Three practical obstacles changed the experiments. A dependency conflict — pandas 3.0 eagerly imports pyarrow, whose OpenMP runtime prevents torch from loading — cost real time and is now documented in the repository so it cannot recur. More substantively, the BiLSTM was initially 85 minutes per run, which would have made the grid impossible; length-bucketed batching cut epoch time roughly fivefold, and reducing hidden size and the token window brought it into budget. That last compromise is a limitation, so rather than bury it, §4.4 measures what it costs and applies the identical cap to both architectures so the comparison stays fair. Reporting the price of a constraint is more useful than pretending it was a free choice.

5.5 Attribution needs interventions, not correlations

The descriptive statistics in §4.2 were satisfying and nearly led to an overclaim. Claude uses far more markdown than the others, so it was tempting to conclude the classifier is substantially a markdown detector. The ablation says otherwise: removing markdown entirely costs only a few points, because the cues are redundant and the model simply falls back on others. Measuring a difference between groups and measuring what a model *depends on* are different questions, and only the second requires retraining under intervention.

5.6 Limitations and what I would do next

- **Period snapshot.** The battles are from 2023 to early 2024, so the findings describe those checkpoints. Model families have changed since, and the fingerprints presumably have too.
- **Family labels pool heterogeneous systems.** The GPT class spans GPT-3.5 and several GPT-4 checkpoints, which caps achievable accuracy; a hierarchical label (family, then checkpoint) would be more informative.
- **Arena prompts are not representative** of ordinary LLM use, being skewed toward tests and probes.
- **Next steps.** Fine-tuning a pretrained encoder would almost certainly beat both from-scratch models and would settle whether the ceiling here is the data or the representation; adding the excluded Gemini family as a sixth class would test whether accuracy degrades gracefully; and an explicit leave-one-

checkpoint-out evaluation would show whether a family fingerprint generalises to a checkpoint never seen in training, which is the version of the question that matters for real forensic use.

Appendix: reproducing these numbers

Every figure and table is generated from the JSON files in `results/`, which are produced by the scripts listed in the repository README. All runs are seeded (448) and execute on CPU. `src/report/build_report.py` reads those files and renders this document, so no number in the report was transcribed by hand.

run	arch	view	transform	test acc	sec
abl_cnn_fixed_length	cnn	output	fixed_length	0.456	32
abl_cnn_function_words	cnn	output	function_words	0.466	68
abl_cnn_no_markdown	cnn	output	no_markdown	0.482	77
abl_cnn_none	cnn	output	none	0.495	76
abl_cnn_shuffled	cnn	output	shuffled	0.408	76
abl_lstm_fixed_length	lstm	output	fixed_length	0.499	161
abl_lstm_no_markdown	lstm	output	no_markdown	0.495	457
cnn_both_s448	cnn	both	none	0.497	134
cnn_input_s448	cnn	input	none	0.204	32
cnn_output_s448	cnn	output	none	0.495	93
cnn_output_s449	cnn	output	none	0.477	84
cnn_output_s450	cnn	output	none	0.457	69
lstm_both_s448	lstm	both	none	0.500	995
lstm_input_s448	lstm	input	none	0.218	137
lstm_output_s448	lstm	output	none	0.511	1119
lstm_output_s449	lstm	output	none	0.515	635
rq3_cnn_cross	cnn	output	none	0.449	228
rq3_cnn_indomain	cnn	output	none	0.511	222
rq3_lstm_cross	lstm	output	none	0.480	392
rq3_lstm_indomain	lstm	output	none	0.508	414
win_cnn_160	cnn	output	none	0.503	159
win_cnn_320	cnn	output	none	0.507	238
win_cnn_40	cnn	output	none	0.462	212
win_cnn_640	cnn	output	none	0.477	193
win_cnn_80	cnn	output	none	0.488	243